```
module counter_divider_top (
    input CLOCK_50,       // 50 MHz clock
    input KEY0,           // Reset button (active LOW)
    output [6:0] HEX0     // 7-segment display
);

    wire clk_1hz;
    wire [2:0] count;

    // Clock Divider: 50 MHz to 1 Hz
    clk_div #(50_000_000) U1 (
        .clk(CLOCK_50),
        .rst(KEY0),
        .clk_out(clk_1hz)
    );

    // 3-bit Counter
    counter U2 (
        .clk(clk_1hz),
        .rst(KEY0),
        .count(count)
    );

    // 7-segment decoder
    segment7 U3 (
        .bcd({1'b0, count}),   // convert 3-bit to 4-bit
        .seg(HEX0)
    );

endmodule
```

```
module clk_div #(parameter DIV = 50_000_000)(
    input clk,
    input rst,
    output reg clk_out
);

    reg [31:0] count;

    always @(posedge clk or negedge rst) begin
        if (!rst) begin
            count <= 0;
            clk_out <= 0;
        end
        else if (count == (DIV/2 - 1)) begin
            count <= 0;
            clk_out <= ~clk_out;
        end
        else begin
            count <= count + 1;
        end
    end

endmodule

module counter(
    input clk,
    input rst,
    output reg [2:0] count
);

    always @(posedge clk or negedge rst) begin
```

```
        if (!rst)
            count <= 3'b000;
        else
            count <= count + 1'b1;
    end

endmodule

module segment7(
    input [3:0] bcd,
    output reg [6:0] seg
);

    always @(*) begin
        case (bcd)
            4'd0: seg = 7'b1000000;
            4'd1: seg = 7'b1111001;
            4'd2: seg = 7'b0100100;
            4'd3: seg = 7'b0110000;
            4'd4: seg = 7'b0011001;
            4'd5: seg = 7'b0010010;
            4'd6: seg = 7'b0000010;
            4'd7: seg = 7'b1111000;
            4'd8: seg = 7'b0000000;
            4'd9: seg = 7'b0010000;
            default: seg = 7'b1111111;
        endcase
    end

endmodule
```